How T3000 reads and saves data

Every value on screen arrived by one of two routes: it was read from the network, or it was loaded from a database. This page follows one round trip — a scan, a panel chosen in the tree, the points it returns, and where the values then live.

Finding the panels

A panel has to be discovered before it can be read. Both protocols are broadcast: the host sends one packet to everything on the segment and collects whatever answers.

Protocol	What the host sends	What a panel answers
Modbus	`FF 19 FE 01 60 57`	`FF 19`, then its Modbus ID, its serial number and a CRC
BACnet	`64 00 00 00 00 00`	`65`, then the device structure

The answer is what fills in the device record T3000 keeps for every panel it has met:

C++

```
struct refresh_net_device
{
    DWORD          nSerial;              // the identity T3000 uses everywhere
    int            modbusID;
    unsigned short product_id;
    CString        ip_address;
    int            nport;
    float          sw_version;
    float          hw_version;
    unsigned int   object_instance;      // BACnet
    unsigned char  panal_number;
    DWORD          parent_serial_number; // set on an expansion module
    CString        NetCard_Address;
    CString        show_label_name;
    unsigned short bacnetip_port;
};
```

Scanning can start by itself when T3000 opens, or on demand from the tree. Automatic scanning is what makes a panel appear without anyone typing an address; a manual scan is what you reach for when a panel has been replaced and kept its address.

The tree

The devices that answered are listed in the system tree. Clicking one is the request that follows: T3000 asks that panel for its inputs, its outputs, its variables, and the rest of its table — and the values arrive in one block rather than one point at a time.

That is the reason a large panel takes a moment to open and then fills in quickly: the read is per panel, not per point.

The read itself

When the panel is already on screen and a value is wanted again, T3000 reads it from the copy it holds in memory rather than going back to the wire. The memory copy is refreshed on demand — by reopening the screen, by a periodic refresh, or by writing a value and reading it back.

This is worth knowing when a reading looks stale: the screen shows what was last read, and the panel's own display may already have moved on.

Where the values are kept

There are three places a value can be, and they answer different questions:

Place	Survives a restart	Holds
the panel	yes — it is the truth	the point's actual value
memory in T3000	no	what was read most recently, for the screens that are open
the database	yes	the points, names, ranges and settings that make the panel usable

Saving to the database is what makes a database project — the panel's points, their names and their configuration — without needing the panel to be online. A panel can be built up in the database first and written to the hardware later.

A `.prg` file is the third form: a single file holding a panel's database record, written and read back from the tree. It is how a panel's setup travels between machines when the panel itself is not present.

The whole picture

The chart below is the same route as a diagram, from the scan to the two stores:

Data flow chart (PDF)

Related

- PTP remote connection — the same read, over a connection that crosses the internet.
- Building Database — the screens that edit the stored copy.
- PRG Files — the file form of a panel's record.